DEEPAK KATUKURI — WEB DESIGN & DEVELOPMENT PORTFOLIO

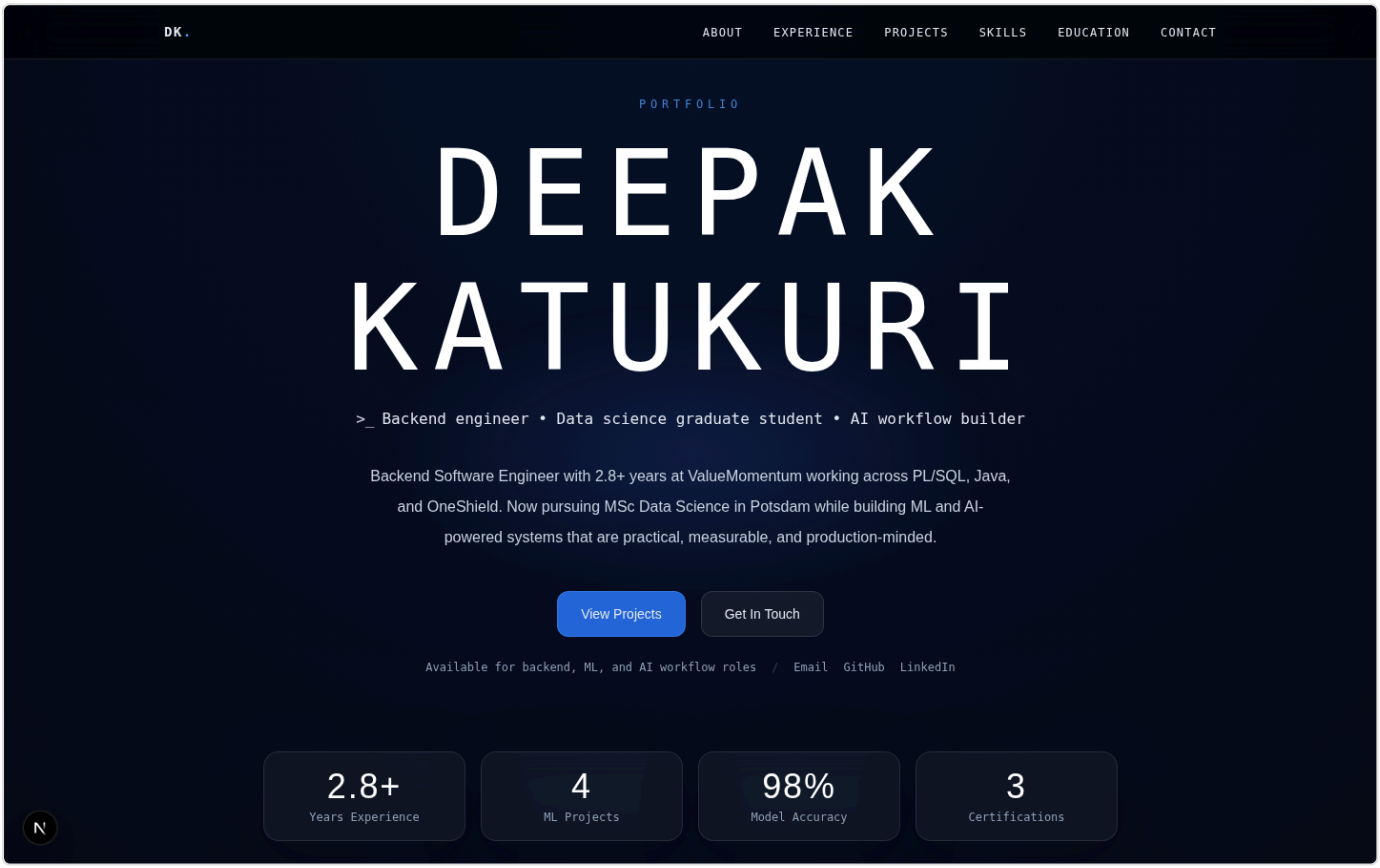
deepakkatukuri@gmail.com · github.com/Deepak589 · Potsdam (Berlin area), Germany

Two builds below: a personal portfolio site (design + frontend, shown in full) and a health-app frontend (product UI, described – screenshots not yet public). Source for both is linked; the portfolio repo is public and cloneable.

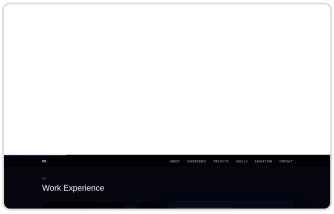
1. PERSONAL PORTFOLIO SITE

Next.js 15 · React 19 · TypeScript · Tailwind CSS · Framer Motion · github.com/Deepak589/NextJs-Portfolio

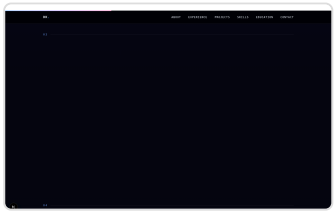
Fully custom single-page site built from scratch – no template. Animated hero with a decode-style text reveal, an interactive orbital timeline for work history, tilting project cards, and a consistent glassmorphism design system carried through every section. Built as reusable components (button, card, badge, radial timeline, tilt card, animated text) rather than one-off page markup.



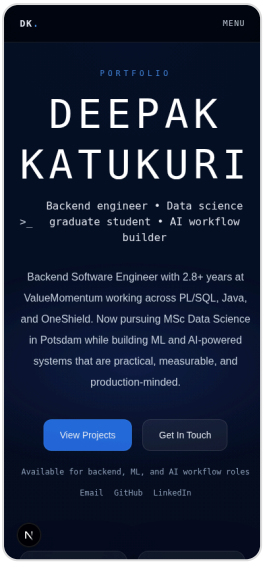
Hero section – animated name reveal, live stat tiles, glassmorphism cards



Interactive orbital timeline – work history



Tilting project cards



Responsive & component system

- Tested and adjusted layout across 390px–1440px breakpoints; nav collapses to a mobile menu, grid sections restack cleanly.
- Skills section (right) groups the stack into scannable categories inside a shared glass-card component.
- Scroll-progress bar, intersection-observer reveal animations, and a consistent type/color system throughout.

> While preparing this application I found and fixed a real bug in the hero's text-reveal animation: a React state/interval race meant the scramble-decode effect could overlap itself and never settle into clean text. Fixed by moving the animation's step-tracking out of React state and into a single persistent timer – verified by rebuilding and re-testing until the text reliably settles.

2. RA NUTRITION & LIFESTYLE APP – FRONTEND

React · Vite · Tailwind CSS · React Query · Recharts · personal product build (screenshots not yet public)

A full-stack health-tracking product with a React/Vite/Tailwind frontend over a FastAPI + PostgreSQL backend. On the frontend specifically:

- Built a reusable Tailwind component system used across every screen instead of one-off styling per page.
- Built a data-dense weekly analytics dashboard (React Query + Recharts) for meal/symptom trends, with explicit empty, loading, and error states rather than leaving those unhandled.
- Wired the UI to JWT-authenticated API flows (login, token refresh, protected routes).